

IBM Project

Report

On

Movie Review Sentiment Analysis - using Modern Architecture

Developed By: -

Patel Maharshi (23162172007)

NITISH SAMIT MARKER (23162582009)

PATEL BHAVIN MAHENDRABHAI (23162582002)

Guided By: -

Dr Sheetal pandya (Internal)

Mr. Palwinder Singh (External)

Submitted to

Faculty of Engineering and Technology

Institute of Computer Technology

Ganpat University



Year - 2026

ABSTRACT

Movie Hub is a full-stack AI-powered web application designed to enhance the movie review experience through intelligent sentiment analysis and personalized user interaction. The platform enables users to create secure accounts, browse real-time movie data fetched from the OMDb API, write reviews, and receive automatically generated ratings based on a trained machine learning model.

The system integrates Flask as the backend framework, SQLAlchemy for database management, and Flask-Login for secure authentication and session handling. User credentials are securely stored using password hashing mechanisms.

A custom-trained AI sentiment model analyzes user-submitted reviews and predicts ratings between 1 and 10. Based on the predicted score, the system classifies reviews into Positive, Neutral, or Negative categories.

TABLE OF CONTENTS

I. Introduction	iv
II. Project Scope	v
III. Software and Hardware Requirements	vi
IV. Process Model	vii
V. Project Plan	viii
VI. Implementation Details	ix
VII. Conclusion	x
VIII. References	xi
IX. Working and Logic Explanation	xii

CHAPTER I: INTRODUCTION

1.1 Overview

- Create secure user accounts
- Browse real-time movie data via OMDb API
- Write reviews
- Receive AI-generated ratings
- View personalized review history
- Analyze sentiment (Positive / Neutral / Negative)
- Integration of Machine Learning, Web Development, Database Systems, API Integration and Authentication

1.2 Motivation

- Automatically analyze review text
- Generate AI-based rating (1–10)
- Classify sentiment automatically
- Store structured relational data
- Demonstrate NLP and API-driven architecture

CHAPTER II: PROJECT SCOPE

- User registration and login
- Secure password hashing
- Persistent database storage
- OMDb API integration
- AI review sentiment analysis
- Rating aggregation
- Dashboard with review history
- Movie-to-review relational mapping
- Personalized user experience

CHAPTER III: SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Python 3.12 – Backend language
- Flask – Web framework
- SQLAlchemy – ORM
- Flask-Login – Authentication
- SQLite – Database
- Joblib – Load ML model
- Scikit-learn – Model training
- Jinja2 – Template rendering
- HTML/CSS – Frontend
- OMDb API – Movie data source
- VS Code – Development environment

Hardware Requirements

- Processor – Intel i3 or higher
- RAM – 4GB minimum
- Storage – 1GB free space
- OS – Windows 10 / Linux / macOS

CHAPTER IV: PROCESS MODEL

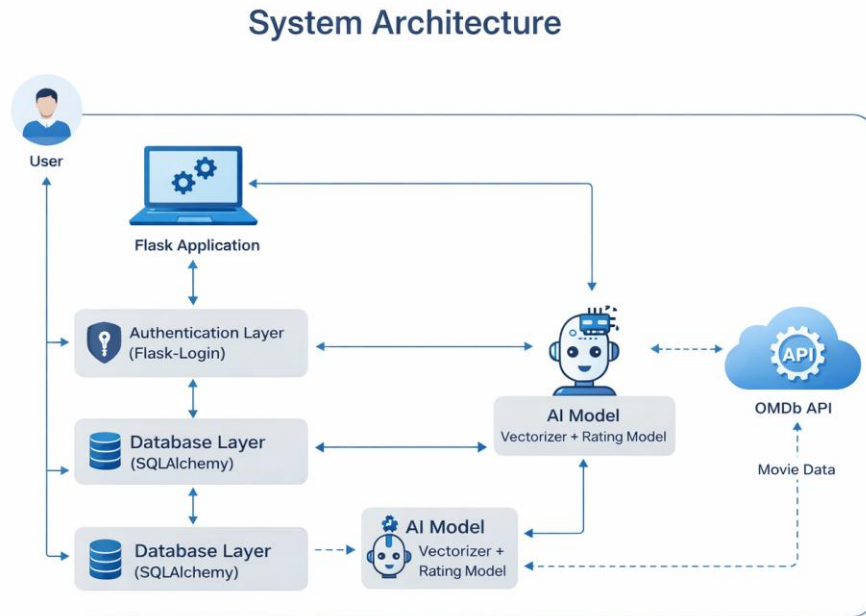
Development Model Used: Incremental Model

- Authentication system
- Database integration
- OMDb API integration
- AI model embedding
- Review storage & aggregation
- Dashboard enhancement
- UI enhancement
- Testing and Debugging

CHAPTER VI: IMPLEMENTATION DETAILS

System Architecture

User → Flask App → Authentication Layer → Database Layer → AI Model → OMDb API



Database Design

- User Table, Movie Table, Review Table with Foreign Key relationships.

Authentication System

- Implemented using Flask-Login with password hashing and session management.

AI Sentiment Model

- Loads rating_model.pkl and vectorizer.pkl
- Vectorizes review text
- Predicts score 1–10
- Classifies sentiment based on threshold

CHAPTER VII: CONCLUSION

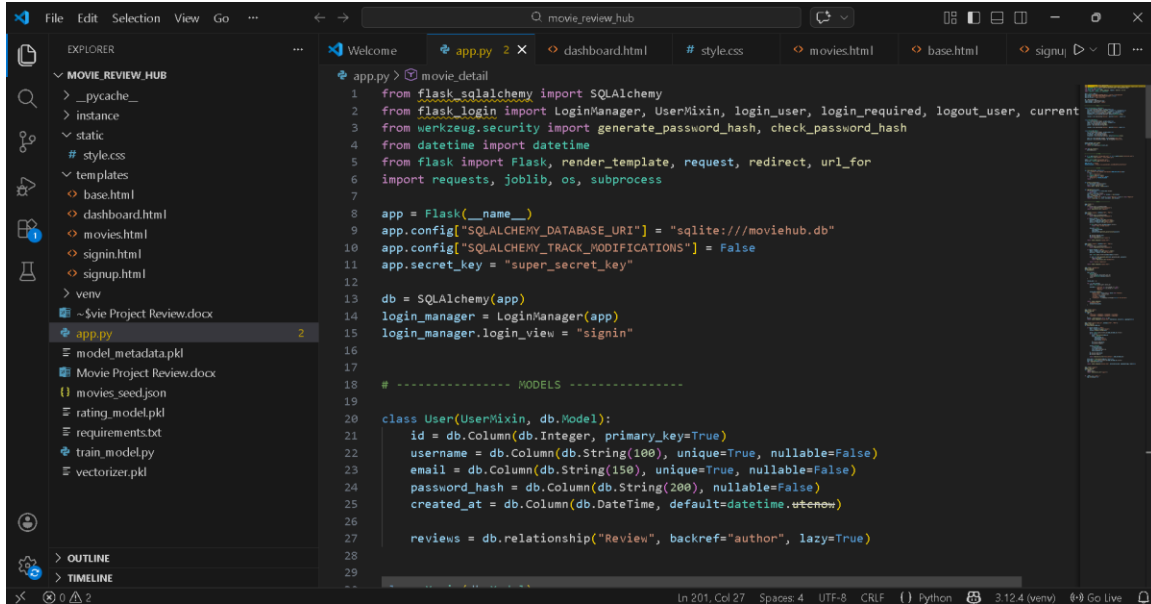
Movie Hub successfully integrates Web Development, Machine Learning, Database Management, API Integration and Secure Authentication into a scalable full-stack system.

CHAPTER VIII: REFERENCES

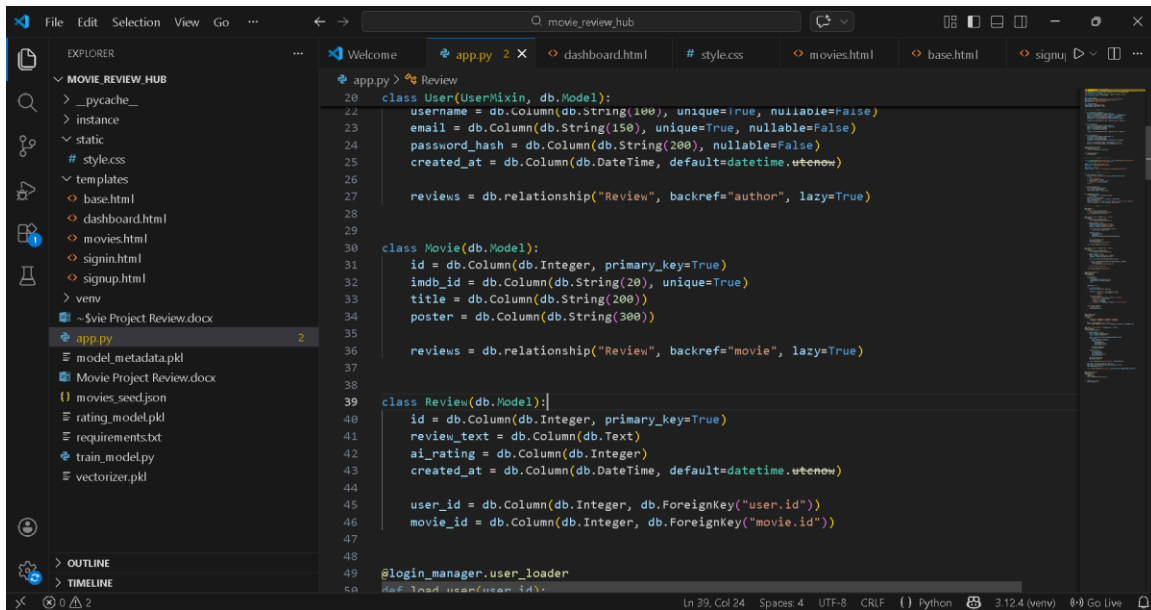
1. Flask Documentation – <https://flask.palletsprojects.com>
2. SQLAlchemy Documentation – <https://docs.sqlalchemy.org>
3. Flask-Login Documentation – <https://flask-login.readthedocs.io>
4. OMDb API – <https://www.omdbapi.com>
5. Scikit-learn Documentation – <https://scikit-learn.org>

CHAPTER IX: WORKING AND LOGIC EXPLANATION

The system follows structured flow from user interaction to database storage and AI-based sentiment prediction. Each module interacts sequentially ensuring secure authentication, data processing and review aggregation.



```
1 from flask_sqlalchemy import SQLAlchemy
2 from flask_login import LoginManager, UserMixin, login_user, login_required, logout_user, current_user
3 from werkzeug.security import generate_password_hash, check_password_hash
4 from datetime import datetime
5 from flask import Flask, render_template, request, redirect, url_for
6 import requests, joblib, os, subprocess
7
8 app = Flask(__name__)
9 app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///moviehub.db"
10 app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
11 app.secret_key = "super_secret_key"
12
13 db = SQLAlchemy(app)
14 login_manager = LoginManager(app)
15 login_manager.login_view = "signin"
16
17 # ----- MODELS -----
18
19 class User(UserMixin, db.Model):
20     id = db.Column(db.Integer, primary_key=True)
21     username = db.Column(db.String(100), unique=True, nullable=False)
22     email = db.Column(db.String(150), unique=True, nullable=False)
23     password_hash = db.Column(db.String(200), nullable=False)
24     created_at = db.Column(db.DateTime, default=datetime.utcnow)
25
26     reviews = db.relationship("Review", backref="author", lazy=True)
```



```
20 class User(UserMixin, db.Model):
21     username = db.Column(db.String(100), unique=True, nullable=False)
22     email = db.Column(db.String(150), unique=True, nullable=False)
23     password_hash = db.Column(db.String(200), nullable=False)
24     created_at = db.Column(db.DateTime, default=datetime.utcnow)
25
26     reviews = db.relationship("Review", backref="author", lazy=True)
27
28 class Movie(db.Model):
29     id = db.Column(db.Integer, primary_key=True)
30     imdb_id = db.Column(db.String(20), unique=True)
31     title = db.Column(db.String(200))
32     poster = db.Column(db.String(300))
33
34     reviews = db.relationship("Review", backref="movie", lazy=True)
35
36 class Review(db.Model):
37     id = db.Column(db.Integer, primary_key=True)
38     review_text = db.Column(db.Text)
39     ai_rating = db.Column(db.Integer)
40     created_at = db.Column(db.DateTime, default=datetime.utcnow)
41
42     user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
43     movie_id = db.Column(db.Integer, db.ForeignKey("movie.id"))
44
45 @login_manager.user_loader
46 def load_user(user_id):
```

```
57 # ----- AI MODEL -----
58
59
60 if not os.path.exists("rating_model.pkl") or not os.path.exists("vectorizer.pkl"):
61     subprocess.run(["python", "train_model.py"])
62
63 model = joblib.load("rating_model.pkl")
64 vectorizer = joblib.load("vectorizer.pkl")
65
66 OMDb_API_KEY = "thetwdb"
67 POSTER_FALLBACK = "https://via.placeholder.com/300x450?text=No+Poster"
68
69
70 # ----- HELPERS -----
71
72
73 def fetch_movie(mid, full=False):
74     url = f"https://www.omdbapi.com/?apikey={OMDb_API_KEY}&i={mid}"
75     if full:
76         url += "&plot=full"
77     r = requests.get(url).json()
78     if r.get("Poster") in ["N/A", None]:
79         r["Poster"] = POSTER_FALLBACK
80     return r
81
82
83 def predict_rating(text):
84     X = vectorizer.transform([text])
85     score = model.predict(X)[0]
86     return max(1, min(10, round(score)))
```

```
1 import joblib
2 import random
3 import numpy as np
4 from sklearn.feature_extraction.text import TfidfVectorizer
5 from sklearn.linear_model import LinearRegression
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import mean_squared_error, r2_score
8
9
10 BASE_DATA = [
11     ("Amazing movie I loved it", 9),
12     ("Great acting and story", 8),
13     ("Masterpiece brilliant film", 10),
14     ("Outstanding direction and acting", 9),
15     ("Very good enjoyable movie", 8),
16     ("Loved every minute of it", 9),
17     ("It was okay not great", 6),
18     ("Average movie slow in parts", 5),
19     ("Decent but could be better", 6),
20     ("One time watch", 5),
21     ("Bad movie boring plot", 3),
22     ("Terrible waste of time", 2),
23     ("Worst movie ever", 1),
24     ("I hate this film", 2),
25     ("Poor acting and bad script", 3)
26 ]
27
28 POS = [
29     "absolutely loved it", "fantastic", "brilliant", "highly recommended",
30     "great storytelling", "amazing visuals", "emotionally powerful"
```

Movie Hub

HomeSign InSign Up

Sign In

Access your personalized Movie Hub

Email address

qwerty@gmail.com

Password

Continue

New here? Create account

Movie Hub

HomeDashboardLogout

Welcome back, 🙌

Here's a snapshot of your AI-powered review journey.

Browse Movies

Movie Hub

HomeDashboardLogout

Your Review History

Track your past reviews and sentiment trends.

6 Review(s)

The Dark Knight

16 Feb 2026 • 07:39 AM

excellent movie

★★★★★

5 / 10 • Neutral

